

Task on RDS

1. Create MariaDB DB on EC2.

Objective

To install and configure a self-managed MariaDB database server on an Amazon EC2 instance.

Purpose

This establishes our **source database**. Later, we will migrate this database from EC2 to Amazon RDS for MariaDB.

Steps:

Step 1 — Update packages

```
sudo dnf update -y
```

Step 2 — Install MariaDB

```
sudo dnf install mariadb105-server -y
```

Verify: `mariadb --version` , `mysql --version`

`mysql --version`

```
sudo systemctl start mariadb
```

```
sudo systemctl status mariadb
```

```
sudo systemctl enable mariadb
```

Step 6 — Secure MariaDB

```
sudo mysql_secure_installation
```

Step 7 — Login

```
sudo mysql -u root -p
```

Step 8 — Verify

```
SELECT VERSION();
```

Task on RDS

Implementation:

```
root@ip-172-31-81-8:/home/ec2-user
[ec2-user@ip-172-31-81-8 ~]$ sudo su
[root@ip-172-31-81-8 ec2-user]# yum -y install mariadb-server
Amazon Linux 2023 Kernel Livepatch repository
No match for argument: mariadb-server
Error: Unable to find a match: mariadb-server
[root@ip-172-31-81-8 ec2-user]# sudo dnf install mariadb105-server -y
Last metadata expiration check: 0:00:31 ago on Wed Sep 16 10:21:04 2026.
Dependencies resolved.

=====
Package                                Architecture      Version                                Repository
=====
Installing:
mariadb105-server                      x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
Installing dependencies:
mariadb-connector-c                    x86_64            3.3.10-1.amzn2023.0.2                amazonlinux
mariadb-connector-c-config             noarch            3.3.10-1.amzn2023.0.2                amazonlinux
mariadb105                             x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
mariadb105-common                     x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
mariadb105-errmsg                      x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
mysql-selinux                          noarch            1.0.14-2.amzn2023.0.1                amazonlinux
perl-DBD-MariaDB                      x86_64            1.22-1.amzn2023.0.5                  amazonlinux
perl-DBI                              x86_64            1.652-1.amzn2023.0.1                 amazonlinux
perl-File-Copy                         noarch            2.34-477.amzn2023.0.9                amazonlinux
perl-Math-BigInt                       noarch            1:1.9998.39-2.amzn2023.0.2           amazonlinux
perl-Math-BigRat                       noarch            0.2624-500.amzn2023.0.2              amazonlinux
perl-Math-Complex                      noarch            1.59-477.amzn2023.0.9                amazonlinux
perl-Module-Load                       noarch            1:0.36-2.amzn2023.0.2                amazonlinux
perl-Sys-Hostname                      x86_64            1.23-477.amzn2023.0.9                amazonlinux
Installing weak dependencies:
mariadb105-backup                     x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
mariadb105-cracklib-password-check    x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
mariadb105-gssapi-server              x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
mariadb105-server-utils               x86_64            3:10.5.29-1.amzn2023.0.1             amazonlinux
=====
Transaction Summary
=====
Install 19 Packages

Total download size: 20 M
Installed size: 117 M
Downloading Packages:
(1/19): mariadb-connector-c-config-3.3.10-1.amzn2023.0.2.noarch.rpm        298 kB/s | 9.3
(2/19): mariadb-connector-c-3.3.10-1.amzn2023.0.2.x86_64.rpm              5.4 MB/s | 211
(3/19): mariadb105-10.5.29-1.amzn2023.0.1.x86_64.rpm                     30 MB/s | 1.5
(4/19): mariadb105-common-10.5.29-1.amzn2023.0.1.x86_64.rpm              973 kB/s | 28
(5/19): mariadb105-cracklib-password-check-10.5.29-1.amzn2023.0.1.x86_64.rpm 495 kB/s | 13
(6/19): mariadb105-errmsg-10.5.29-1.amzn2023.0.1.x86_64.rpm              6.7 MB/s | 212
(7/19): mariadb105-gssapi-server-10.5.29-1.amzn2023.0.1.x86_64.rpm        512 kB/s | 15
(8/19): mariadb105-backup-10.5.29-1.amzn2023.0.1.x86_64.rpm              56 MB/s | 6.0
(9/19): mariadb105-server-utils-10.5.29-1.amzn2023.0.1.x86_64.rpm         5.5 MB/s | 207
(10/19): mysql-selinux-1.0.14-2.amzn2023.0.1.noarch.rpm                   1.3 MB/s | 36
(11/19): perl-DBD-MariaDB-1.22-1.amzn2023.0.5.x86_64.rpm                 5.4 MB/s | 152
(12/19): perl-File-Copy-2.34-477.amzn2023.0.9.noarch.rpm                  766 kB/s | 20
(13/19): perl-DBI-1.652-1.amzn2023.0.1.x86_64.rpm                        19 MB/s | 811
(14/19): perl-Math-BigInt-1.9998.39-2.amzn2023.0.2.noarch.rpm             7.0 MB/s | 202
(15/19): mariadb105-server-10.5.29-1.amzn2023.0.1.x86_64.rpm             52 MB/s | 10
(16/19): perl-Math-BigRat-0.2624-500.amzn2023.0.2.noarch.rpm              497 kB/s | 42
(17/19): perl-Math-Complex-1.59-477.amzn2023.0.9.noarch.rpm              697 kB/s | 47
(18/19): perl-Sys-Hostname-1.23-477.amzn2023.0.9.x86_64.rpm              825 kB/s | 17
(19/19): perl-Module-Load-0.36-2.amzn2023.0.2.noarch.rpm                  618 kB/s | 18
Total                                                                       52 MB/s | 20
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
```

```
root@ip-172-31-81-8:/home/ec2-user
[root@ip-172-31-81-8 ec2-user]# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SELECT VERSION();
+-----+
| VERSION() |
+-----+
| 10.5.29-MariaDB |
+-----+
1 row in set (0.000 sec)

MariaDB [(none)]> |
```

Conclusion

MariaDB has been successfully installed and configured on the EC2 instance. The EC2 instance is now acting as a self-managed database server and will be used as the source database for the migration exercise.

Task on RDS

2. Insert some dummy data.

Objective

Create a sample database and populate it with dummy employee information.

Purpose

The dummy database gives us actual data that can later be backed up and migrated to Amazon RDS

Steps:

Step 1 — Login

```
sudo mysql -u root -p
```

Step 2 — Create database

```
CREATE DATABASE companydb;
```

Verify:

```
SHOW DATABASES;
```

Step 3 — Select database

```
USE companydb;
```

Step 4 — Create employees table

```
CREATE TABLE employees (  
id INT PRIMARY KEY AUTO_INCREMENT,  
name VARCHAR(100),  
department VARCHAR(100),  
salary DECIMAL(10,2),  
city VARCHAR(100)  
);
```

Step 5 — Insert dummy records

```
INSERT INTO employees (name, department, salary, city)  
VALUES  
( 'John', 'DevOps', 65000, 'New York'),  
( 'David', 'Cloud', 72000, 'Chicago'),  
( 'Sarah', 'Database', 68000, 'Boston'),  
( 'Michael', 'Linux', 60000, 'Seattle'),  
( 'Emma', 'AWS', 75000, 'Dallas');
```

Step 6 — Verify records

```
SELECT * FROM employees;
```

Step 7 — Verify table

```
SHOW TABLES;
```

Step 8 — Exit

```
EXIT;
```

Verification:

```
sudo mysql -u root -p companydb -e "SELECT * FROM employees;"
```

Implementation:

Task on RDS

```
-> ^C
MariaDB [companydb]> INSERT INTO employees (name, department, salary, city)
-> VALUES
-> ('John', 'DevOps', 65000, 'New York'),
-> ('David', 'Cloud', 72000, 'Chicago'),
-> ('Sarah', 'Database', 68000, 'Boston'),
-> ('Michael', 'Linux', 60000, 'Seattle'),
-> ('Emma', 'AWS', 75000, 'Dallas');
Query OK, 5 rows affected (0.003 sec)
Records: 5 Duplicates: 0 Warnings: 0

MariaDB [companydb]> SELECT * FROM employees;
+-----+-----+-----+-----+-----+
| id | name  | department | salary | city  |
+-----+-----+-----+-----+-----+
| 1  | John  | DevOps    | 65000.00 | New York |
| 2  | David | Cloud     | 72000.00 | Chicago |
| 3  | Sarah | Database  | 68000.00 | Boston  |
| 4  | Michael | Linux    | 60000.00 | Seattle |
| 5  | Emma  | AWS       | 75000.00 | Dallas  |
+-----+-----+-----+-----+-----+
5 rows in set (0.000 sec)

MariaDB [companydb]> SHOW TABLES;
+-----+
| Tables_in_companydb |
+-----+
| employees            |
+-----+
1 row in set (0.000 sec)

MariaDB [companydb]> █

MariaDB [companydb]> exit
Bye
[root@ip-172-31-81-8 ec2-user]# sudo mysql -u root -p companydb -e "SELECT * FROM employees;"
Enter password:
+-----+-----+-----+-----+-----+
| id | name  | department | salary | city  |
+-----+-----+-----+-----+-----+
| 1  | John  | DevOps    | 65000.00 | New York |
| 2  | David | Cloud     | 72000.00 | Chicago |
| 3  | Sarah | Database  | 68000.00 | Boston  |
| 4  | Michael | Linux    | 60000.00 | Seattle |
| 5  | Emma  | AWS       | 75000.00 | Dallas  |
+-----+-----+-----+-----+-----+
[root@ip-172-31-81-8 ec2-user]# █
```

Conclusion

A sample companydb database has been created on the EC2-hosted MariaDB server and populated with employee records. This database will be used for the backup and RDS migration tasks.

Task on RDS

3. Take the backup of dummy data on EC2.

Objective

Create a logical backup of the MariaDB database using mariadb-dump/mysqldump.

Purpose

A database dump allows us to preserve the database structure and data in an SQL file and use that file for migration or disaster recovery.

Step 1 — Create backup directory

```
mkdir -p ~/db-backups
```

Step 2 — Check database

```
sudo mysql -u root -p -e "SHOW DATABASES;"
```

Step 3 — Take backup

```
sudo mariadb-dump -u root -p \  
--databases companydb \  
--single-transaction \  
--order-by-primary \  
> ~/db-backups/companydb_backup.sql
```

Step 4 — Verify backup

```
ls -lh ~/db-backups/
```

Step 5 — Inspect backup

```
head -30 ~/db-backups/companydb_backup.sql
```

Step 6 — Check backup size

```
du -h ~/db-backups/companydb_backup.sql
```

Optional — Compress backup

```
gzip ~/db-backups/companydb_backup.sql
```

Check:

```
ls -lh ~/db-backups/
```

Verification

```
test -s ~/db-backups/companydb_backup.sql && echo "Backup exists and is not empty"
```

Expected:

Backup exists and is not empty

Implementation:

```
[root@ip-172-31-81-8 ec2-user]#  
[root@ip-172-31-81-8 ec2-user]#  
[root@ip-172-31-81-8 ec2-user]# mkdir -p ~/db-backups  
[root@ip-172-31-81-8 ec2-user]# sudo mysql -u root -p -e "SHOW DATABASES;"  
Enter password:  
+-----+  
| Database |  
+-----+  
| companydb |  
| information_schema |  
| mysql |  
| performance_schema |  
+-----+  
[root@ip-172-31-81-8 ec2-user]# sudo mariadb-dump -u root -p \  
--databases companydb \  
--single-transaction \  
--order-by-primary \  
> ~/db-backups/companydb_backup.sql  
Enter password:  
[root@ip-172-31-81-8 ec2-user]# ls -lh ~/db-backups/  
total 4.0K  
-rw-r--r--. 1 root root 2.5K Sep 16 11:00 companydb_backup.sql
```

Task on RDS

```
rw-r--r-- 1 root root 215K Sep 10 11:00 companydb_backup.sql
[root@ip-172-31-81-8 ec2-user]# head -30 ~/db-backups/companydb_backup.sql
/*M!999999\~ enable the sandbox mode */
-- MariaDB dump 10.19 Distrib 10.5.29-MariaDB, for Linux (x86_64)
--
-- Host: localhost    Database: companydb
--
-----
-- Server version      10.5.29-MariaDB

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

--
-- Current Database: `companydb`
--

CREATE DATABASE /*!32312 IF NOT EXISTS*/ `companydb` /*!40100 DEFAULT CHARACTER SET latin1 COLLATE latin1_swedish_ci */;

USE `companydb`;

--
-- Table structure for table `employees`
--

[root@ip-172-31-81-8 ec2-user]# du -h ~/db-backups/companydb_backup.sql
4.0K    ~/db-backups/companydb_backup.sql
[root@ip-172-31-81-8 ec2-user]# test -s ~/db-backups/companydb_backup.sql && echo "Backup exists and is not empty"
Backup exists and is not empty
[root@ip-172-31-81-8 ec2-user]#

[root@ip-172-31-81-8 ec2-user]# mariadb-dump --version
mariadb-dump Ver 10.19 Distrib 10.5.29-MariaDB, for Linux (x86_64)
[root@ip-172-31-81-8 ec2-user]#
```

Conclusion:

A logical backup of the companydb MariaDB database has been successfully created on the EC2 instance. The SQL backup contains the database structure and dummy data and can be used for migration into Amazon RDS.

Task on RDS

4. Launch MariaDB RDS instance.

Objective

Create a managed Amazon RDS for MariaDB database instance.

Purpose

Instead of managing the MariaDB operating system, patches, database host, and infrastructure ourselves, RDS manages the database infrastructure.

AWS describes RDS DB instances as managed database environments and supports MariaDB and MySQL DB engines

Steps:

AWS CONSOLE NAVIGATION :

[AWS Console](#) ↓ [Search "RDS"](#) ↓ [Amazon RDS](#) ↓ [Databases](#) ↓ [Create database](#) → Full Configuration

Step 1 — Choose creation method

[Standard create](#)

Step 2 — Engine

[Select: MariaDB](#)

Step 3 — Template

[Dev/Test](#)

Step 4 — Settings

[DB instance identifier: companydb-mariadb-rds](#)

[Master username: admin](#)

Step 5 — Instance configuration

[Choose small instance](#)

Step 6 — Storage

[General Purpose SSD](#)

Step 7 — Connectivity

[VPC: Same VPC as EC2](#)

Step 8 — Security group : rds-mariadb-sg

[Type: MYSQL/Aurora](#)

[Protocol: TCP](#)

[Port: 3306](#)

[Source: EC2 security group](#)

Step 9 — Database authentication

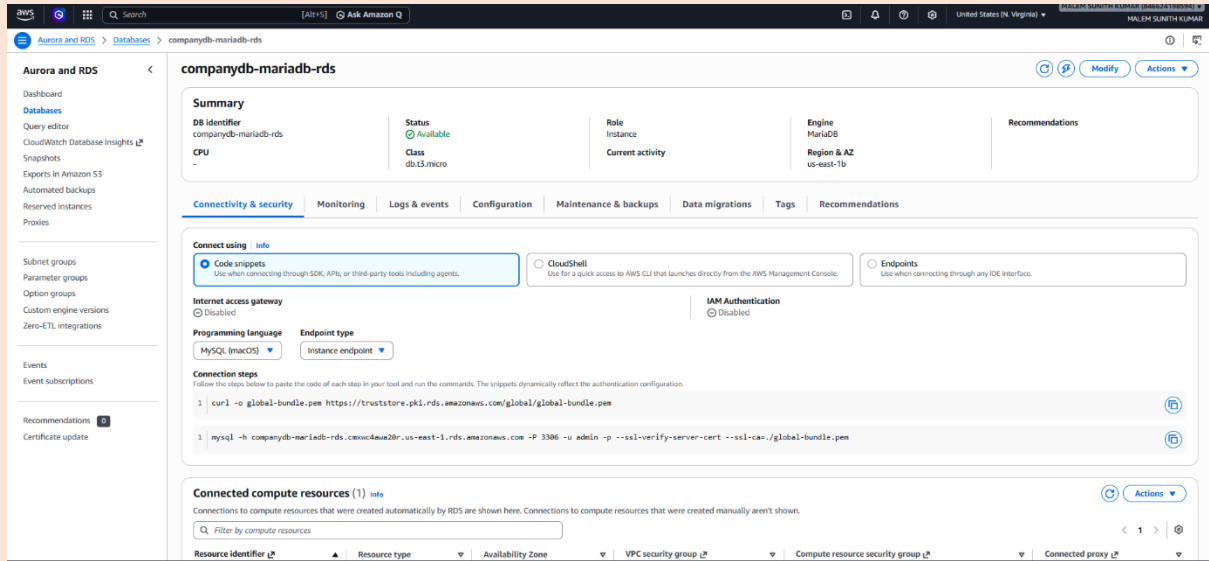
Step 10 — Additional configuration

[Database name: companydb](#)

Step 11 — Create database

Task on RDS

Implementation:



Conclusion:

An Amazon RDS for MariaDB DB instance has been successfully created. The RDS instance is now ready to receive the database migrated from the EC2 MariaDB server.

5. Migrate database from EC2 to RDS.

Objective

Migrate companydb from the self-managed MariaDB server on EC2 into Amazon RDS for MariaDB.

Purpose

This demonstrates a common cloud migration pattern:

Self-managed EC2 MariaDB ↓ logical dump ↓ RDS MariaDB

Steps:

Step 1 — Test network connectivity

```
nc -zv companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com 3306
```

If nc is not installed:

```
sudo dnf install nc -y
```

Step 2 — Connect to RDS

```
mariadb \  
-h companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.\  
-P 3306 \  
-u admin \  
-p
```

Step 3 — Check databases

```
SHOW DATABASES;
```

```
EXIT;
```

Task on RDS

Step 4 — Create Dump File

```
mariadb-dump \  
-u root \  
-p \  
--databases companydb \  
--single-transaction \  
--order-by-primary \  
--routines=0 \  
--triggers=0 \  
--events=0 \  
> companydb.sql
```

Then check:

ls -lh companydb.sql

Step 5 — Download the RDS certificate

```
curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

Step 5 — Connect using SSL

```
mariadb -h companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com -P 3306 -u admin  
-p --ssl-verify-server-cert --ssl-ca=./global-bundle.pem
```

import into RDS

```
mariadb \  
-h companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com \  
-P 3306 \  
-u admin \  
-p \  
--ssl-verify-server-cert \  
--ssl-ca=./global-bundle.pem \  
< companydb.sql
```

Enter password:

This time enter ONLY the RDS password

Then Verify Migration

```
mariadb \  
-h companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com \  
-P 3306 \  
-u admin \  
-p \  
--ssl-verify-server-cert \  
--ssl-ca=./global-bundle.pem
```

Task on RDS

Implementation:

```
[root@ip-172-31-81-8 ec2-user]# nc -zv companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com 3306
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Connected to 172.31.91.105:3306.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
[root@ip-172-31-81-8 ec2-user]#
```

```
Last login: Wed Sep 16 12:17:39 2026 from 103.160.27.240
[ec2-user@ip-172-31-81-8 ~]$ sudo su
[root@ip-172-31-81-8 ec2-user]# echo test
test
[root@ip-172-31-81-8 ec2-user]# ls -lh global-bundle.pem
-rw-r--r-- 1 root root 162K Sep 16 12:10 global-bundle.pem
[root@ip-172-31-81-8 ec2-user]# mariadb -h companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-verify-server-cert --ssl-ca=./global-bundle.pem
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 103
Server version: 11.8.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

```
[root@ip-172-31-81-8 ec2-user]# mariadb-dump \
-u root \
-p \
--databases companydb \
--single-transaction \
--order-by-primary \
--routines=0 \
--triggers=0 \
--events=0 \
> companydb.sql
Enter password:
[root@ip-172-31-81-8 ec2-user]# ls -lh companydb.sql
-rw-r--r-- 1 root root 2.5K Sep 16 12:22 companydb.sql
[root@ip-172-31-81-8 ec2-user]# mariadb \
-h companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com \
-P 3306 \
-u admin \
-p \
--ssl-verify-server-cert \
--ssl-ca=./global-bundle.pem \
< companydb.sql
Enter password:
[root@ip-172-31-81-8 ec2-user]# mariadb \
-h companydb-mariadb-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com \
-P 3306 \
-u admin \
-p \
--ssl-verify-server-cert \
--ssl-ca=./global-bundle.pem
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 109
Server version: 11.8.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE companydb;
Reading table information for completion of table and column names
You can turn off this feature with -A

Database changed
MariaDB [companydb]> SHOW TABLES;
+-----+
| Tables_in_companydb |
+-----+
| employees            |
+-----+
```

Conclusion

The companydb database has been successfully migrated from the self-managed MariaDB server running on EC2 to Amazon RDS for MariaDB. The database structure and dummy records were verified on the RDS instance.

Task on RDS

6. Install MySQL DB on EC2.

Objective

Install a MySQL-compatible database environment on another EC2 instance or the same lab EC2 instance.

Purpose

This task demonstrates the difference between a self-managed database server on EC2 and a managed MySQL database service on RDS.

Steps:

MySQL 8.4 LTS Installation on Amazon Linux 2023

1. Verify the Operating System

```
cat /etc/os-release
```

Check disk space: `df -h`

2. Check for Existing MariaDB Installation

```
rpm -qa | grep -Ei 'mariadb|mysql'
```

```
systemctl status mariadb --no-pager 2>/dev/null || true
```

3. Stop MariaDB

```
sudo systemctl stop mariadb 2>/dev/null || true
```

Disable it :

```
sudo systemctl disable mariadb 2>/dev/null || true
```

4. Remove MariaDB Packages

```
sudo dnf remove -y 'mariadb*'
```

```
sudo dnf autoremove -y
```

Verify:

```
rpm -qa | grep -Ei 'mariadb'
```

5. Remove Old MariaDB Database Files

```
sudo rm -rf /var/lib/mysql
```

```
sudo rm -f /etc/my.cnf
```

```
sudo rm -rf /etc/my.cnf.d
```

6. Back Up the MySQL Repository Configuration

```
sudo cp -a /etc/yum.repos.d/mysql-community.repo \
/etc/yum.repos.d/mysql-community.repo.backup
```

Also

```
sudo cp -a /etc/yum.repos.d/mysql-community-source.repo \
/etc/yum.repos.d/mysql-community-source.repo.backup
```

7. Configure the MySQL 8.4 LTS Repository

```
sudo dnf install https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm -y
```

8. Install MySQL

```
sudo dnf install mysql-co
```

9. Start MySQL

```
sudo systemctl start mysqld
```

10. Enable MySQL at Boot

```
sudo systemctl enable mysql
```

Verify Status

```
sudo systemctl status mysql
```

Task on RDS

Implementation:

```
Amazon Linux 2023 Kernel Livepatch repository      721 kB/s | 75 kB      00:00
MySQL 8.4 LTS Community Server                    31 MB/s | 2.3 MB     00:00
Metadata cache created.
[root@ip-172-31-95-2 tmp]# sudo dnf info mysql-community-server
Last metadata expiration check: 0:00:35 ago on Wed Sep 16 16:39:44 2026.
Available Packages
Name       : mysql-community-server
Version    : 8.4.11
Release    : 1.el9
Architecture : x86_64
Size       : 52 M
Source     : mysql-community-8.4.11-1.el9.src.rpm
Repository : mysql-8.4-lts-community
Summary    : A very fast and reliable SQL database server
URL        : http://www.mysql.com/
License    : Copyright (c) 2000, 2026, Oracle and/or its affiliates. Under GPLv2
           : license as shown in the Description field.
Description: The MySQL(TM) software delivers a very fast, multi-threaded,
           : multi-user, and robust SQL (Structured Query Language) database server.
           : MySQL Server is intended for mission-critical, heavy-load production
           : systems as well as for embedding into mass-deployed software. MySQL is
           : a trademark of Oracle and/or its affiliates
           :
           : The MySQL software has Dual Licensing, which means you can use the
           : MySQL software free of charge under the GNU General Public License
           : (http://www.gnu.org/licenses/). You can also purchase commercial MySQL
           : licenses from Oracle and/or its affiliates if you do not wish to be
           : bound by the terms of the GPL. See the chapter "Licensing and Support"
           : in the manual for further info.
           :
           : The MySQL web site (http://www.mysql.com/) provides the latest news and
           : information about the MySQL software. Also please see the
           : documentation and the manual for more information.
           :
           : This package includes the MySQL server binary as well as related
           : utilities to run and administer a MySQL server.

[root@ip-172-31-95-2 tmp]# rpm -qa | grep -Ei 'mariadb'
[root@ip-172-31-95-2 tmp]# sudo dnf install -y mysql-community-server
Last metadata expiration check: 0:01:20 ago on Wed Sep 16 16:39:44 2026.
Dependencies resolved.
=====
Package                                Arch      Version      Repository      Size
=====
Installing:
mysql-community-server                 x86_64    8.4.11-1.el9  mysql-8.4-lts-community  52 M
Installing dependencies:
mysql-community-client                 x86_64    8.4.11-1.el9  mysql-8.4-lts-community  3.4 M
mysql-community-client-plugins         x86_64    8.4.11-1.el9  mysql-8.4-lts-community  1.5 M
mysql-community-common                 x86_64    8.4.11-1.el9  mysql-8.4-lts-community  581 k
mysql-community-icu-data-files         x86_64    8.4.11-1.el9  mysql-8.4-lts-community  2.3 M
mysql-community-libs                   x86_64    8.4.11-1.el9  mysql-8.4-lts-community  1.5 M

Transaction Summary
=====
Install 6 Packages
```

Task on RDS

```
Complete!
[root@ip-172-31-95-2 tmp]# rpm -qa | grep -Ei 'mysql|mariadb'
mysql84-community-release-el9-4.noarch
mysql-community-common-8.4.11-1.el9.x86_64
mysql-community-client-plugins-8.4.11-1.el9.x86_64
mysql-community-libs-8.4.11-1.el9.x86_64
mysql-community-client-8.4.11-1.el9.x86_64
mysql-community-icu-data-files-8.4.11-1.el9.x86_64
mysql-community-server-8.4.11-1.el9.x86_64
[root@ip-172-31-95-2 tmp]# sudo systemctl enable mysqld
sudo systemctl start mysqld
[root@ip-172-31-95-2 tmp]# sudo systemctl status mysqld --no-pager
● mysqld.service - MySQL Server
   Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-09-16 16:41:55 UTC; 6s ago
     Docs: man:mysqld(8)
           http://dev.mysql.com/doc/refman/en/using-systemd.html
   Process: 41107 ExecStartPre=/usr/bin/mysqld_pre_systemd (code=exited, status=0/SUCCESS)
   Main PID: 41204 (mysqld)
    Status: "Server is operational"
      Tasks: 35 (limit: 1113)
     Memory: 446.7M
        CPU: 3.394s
    CGroup: /system.slice/mysqld.service
            └─41204 /usr/sbin/mysqld
```

Conclusion

A MySQL database environment has been prepared on EC2. The EC2 environment demonstrates the self-managed model where database software and the underlying operating system are administered by the user.

7. Launch MySQL RDS image.

Objective

Create an Amazon RDS for MySQL database instance.

Purpose

This demonstrates deploying MySQL using the AWS-managed RDS service rather than installing and maintaining MySQL manually on EC2.

Steps:

AWS CONSOLE NAVIGATION:

[AWS Console](#) ↓ [RDS](#) ↓ [Databases](#) ↓ [Create database](#) ↓ [Standard create](#) → [Full Configuration](#)

Step 1 — Engine : [Mysql](#)

Step 2 — Version

Step 3 — Template : [Dev/Test](#)

Step 4 — DB identifier : [companydb](#)

Step 5 — Credentials : [admin](#)

Step 6 — Instance class : [Choose small](#)

Step 7 — Storage : [General Purpose SSD](#)

Step 8 — Connectivity : [Use the same VPC as EC2.](#)

Step 9 — Database name : [companydb](#)

Step 10 — Backup : [Backup retention: 1 day](#)

Step 11 — Create

Task on RDS

Implementation:

Create database Info

Engine options
Engine type Info

☐ Aurora (MySQL Compatible)


☐ Aurora (PostgreSQL Compatible)


☒ MySQL


☐ PostgreSQL


☐ MariaDB


☐ Oracle


☐ Microsoft SQL Server


☐ IBM Db2


companydb-mysql-rds Info Modify Actions

Summary
DB identifier
companydb-mysql-rds
CPU
5.33%

Status
☒ Modifying
Class
db.t3.micro

Role
Instance
Current activity
0 Connections

Engine
MySQL Community
Region & AZ
us-east-1b

Recommendations

Storage initialization in progress
Storage initialization is in progress. Your instance is available and functional, but it may experience increased latency. See the Primary storage or Additional storage volumes (if applicable) section on the Configuration tab for status and progress.

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations | Maintenance & backups | Data migrations | Tags | Recommendations

Connect using Info
☒ Code snippets
Use when connecting through SDK, APIs, or third-party tools including agents.
☐ CloudShell
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
☐ Endpoints
Use when connecting through any IDE interface.

Internet access gateway
☒ Disabled
IAM Authentication
☒ Disabled

Programming language
MySQL (macOS)
Endpoint type
Instance endpoint

Connection steps
Follow the steps below to paste the code of each step in your tool and run the commands. The snippets dynamically reflect the authentication configuration.
1 | `curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem`
1 | `mysql -h companydb-mysql-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem`

Connected compute resources (1) Info Actions
Connections to compute resources that were created automatically by RDS are shown here. Connections to compute resources that were created manually aren't shown.

```
mysql> exit
Bye
[root@ip-172-31-95-2 tmp]# mysql \
-h companydb-mysql-rds.cmxwc4awa20r.us-east-1.rds.amazonaws.com \
-P 3306 \
-u admin \
-p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 28
Server version: 8.4.11 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql>
```

Conclusion

An Amazon RDS for MySQL DB instance has been successfully launched. The MySQL database is now managed by Amazon RDS instead of being managed directly on an EC2 operating system.

Task on RDS

8. Configure Multi-AZ.

Objective

Convert the RDS MySQL DB instance to a Multi-AZ deployment.

Purpose

Multi-AZ provides a standby database instance in another Availability Zone for failover support. The standby in a traditional Multi-AZ DB instance deployment is not used for read traffic.

Steps:

Step 1 — Find Availability section

Availability & durability

Step 2 — Apply

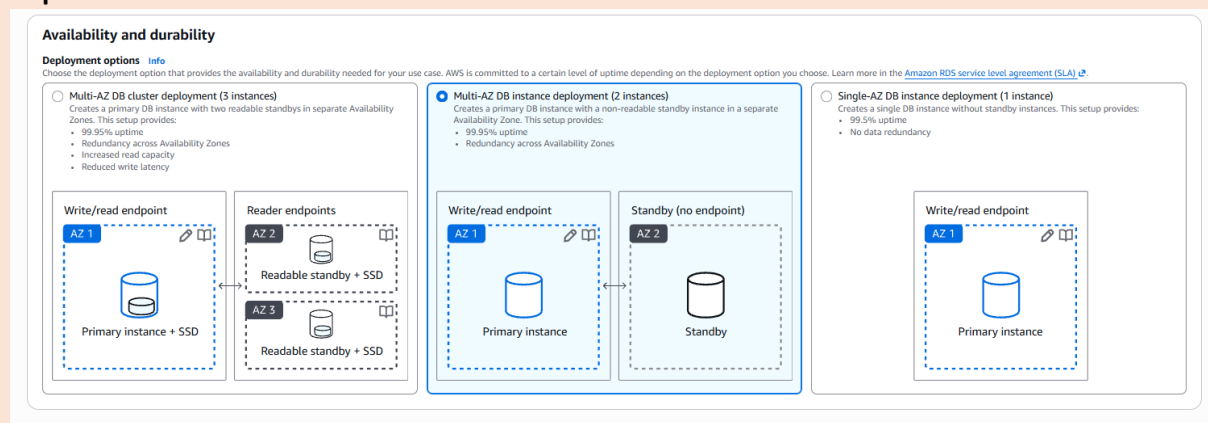
Continue

Choose: [Apply immediately](#)

Step 3 — Wait

Step 4 — Verify

Implementation :



Conclusion :

The RDS MySQL database has been configured as a Multi-AZ deployment. A standby database is maintained in another Availability Zone to provide high availability and automatic failover support. Multi-AZ is intended for availability rather than read scaling.

Task on RDS

9. Take backup of DB and restore the DB.

Objective

Create an RDS DB snapshot and restore it into a new RDS database instance.

Purpose

This demonstrates database backup and disaster-recovery restoration using Amazon RDS.

AWS states that restoring an RDS DB snapshot creates a new DB instance; you cannot restore a snapshot directly over an existing DB instance

Steps:

Step 1 — Snapshot name : [Companydb-snapshot](#)

Step 2 — Create snapshot

Step 3 — Monitor

PART B — RESTORE SNAPSHOT

Step 1 — Select snapshot

[RDS](#) ↓ [Snapshots](#) ↓ [Manual](#) ↓ [companydb-mysql-snapshot-01](#)

Step 2 — Restore

[Actions](#) ↓ [Restore snapshot](#)

Step 3 — New DB identifier

[companydb-mysql-restored](#)

Step 4 — Configure

[VPC Subnet group](#)

[Security group](#)

[Instance class](#)

[Storage](#)

PART C — VERIFY RESTORED DATABASE

`mysql \`

`-h RESTORED-ENDPOINT \`

`-P 3306 \`

`-u admin \`

`-p`

Implementation:

Take DB Snapshot

Preferences
To take a DB Snapshot, choose a database and name your DB Snapshot.

DB instance
DB Instance identifier. This is the unique key that identifies a DB Instance.

Snapshot name
Identifier for the DB Snapshot.

Snapshot identifier is case insensitive, but stored as all lower-case, as in "mysnapshot". Cannot be null, empty, or blank. Must contain from 1 to 255 alphanumeric characters or hyphens. First character must be a letter. Cannot end with a hyphen or contain two consecutive hyphens.

[Cancel](#) [Take snapshot](#)

Conclusion

A manual RDS DB snapshot was created successfully and restored into a new RDS MySQL DB instance. The restored database was verified by connecting to it and checking the database, table, and data.

Task on RDS

10. Create read replica.

Objective

Create a read replica of the RDS MySQL database.

Purpose

A read replica provides a separate DB instance that can be used to serve read traffic and reduce read workload on the primary database.

BEFORE CREATING READ REPLICA

DS ↓ Databases ↓ companydb-mysql-rds ↓ Configuration

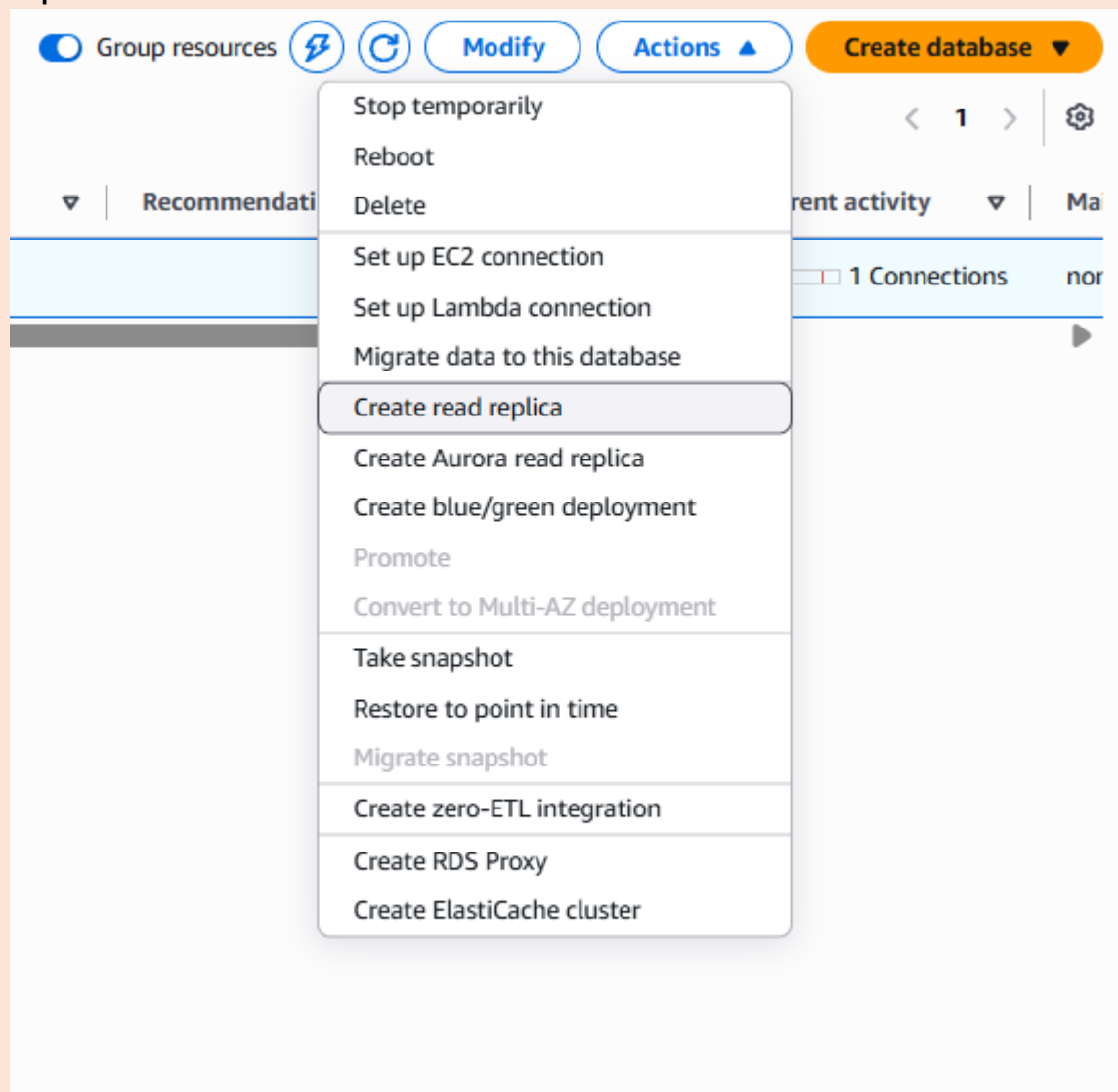
Look for: Backup retention period

It must be greater than: 0

CREATE READ REPLICA

RDS ↓ Databases ↓ companydb-mysql-rds ↓ Actions ↓ Create read replica

Implementation:



Task on RDS

Create read replica

You are creating a replica DB instance from a source DB instance. This new DB instance will have the source DB instance's DB security groups and DB parameter groups.

Settings

Replica source

Source DB instance identifier

companydb-mysql-rds
Role: Instance

DB instance identifier

This is the unique key that identifies a DB instance. This parameter is stored as a lowercase string (for example, mydbinstance).

Compnaydb-read-replica

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class

[Info](#)

▼ Hide filters

☒ Include previous generation classes

☐ Standard classes (includes m classes)

☐ Memory optimized classes (includes r and x classes)

☒ Burstable classes (includes t classes)

Instance type

db.t3.micro

2 vCPUs 1 GiB RAM EBS Bandwidth: Up to 2,085 Mbps Network: Up to 5 Gbps

Availability

Deployment options

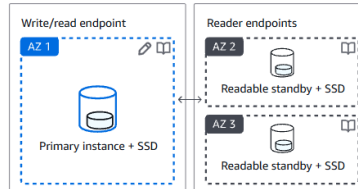
[Info](#)

The following deployment options are limited to those supported by the engine.

☐ Multi-AZ DB cluster deployment (3 instances)

Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:

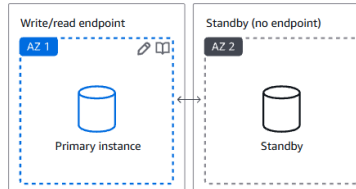
- 99.95% uptime
- Redundancy across Availability Zones
- Increased read capacity
- Reduced write latency



☐ Multi-AZ DB instance deployment (2 instances)

Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:

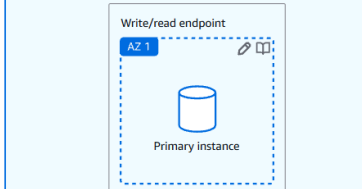
- 99.95% uptime
- Redundancy across Availability Zones



☒ Single-AZ DB instance deployment (1 instance)

Creates a single writer DB instance with no reader instances. This setup provides:

- 99.5% uptime
- No data redundancy



```
bye
[root@ip-172-31-95-2 tmp]# mysql \
-h companydb-mysql-rds.cmxc4awa20r.us-east-1.rds.amazonaws.com \
-P 3306 \
-u admin \
-p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 28
Server version: 8.4.11 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

Conclusion:

The read replica was successfully created from the primary Amazon RDS MySQL database. The replica is deployed in the same AWS Region using a Single-AZ configuration and is independently available for read operations.